

operands on the + sign must be a String. What about this example, where we're concatenating a String object to a regular object. What happens here?

```
String a = "";  
String b = "Test";  
Employee e = new Employee("GW", "Bush");  
a = b + e;  
This still produces a String, and is equivalent to:  
a = b + e.toString();
```

Class String is located at the top of the class hierarchy. The class is declared as final, which means it cannot be Subclassed, cannot be a Superclass to any Subclass. Class String contains 48 methods. These methods allow you to do many things with a String object except change the String. You can do anything with a String except change that String. There are many methods that take a String argument and return a String. None of these allow you to change the original String. These methods may appear to change the String, but they don't. They merely return a different String. Once a String object is instantiated, it can never be changed. Strings are immutable. That means, once they are created, they can't be changed. Because String objects are immutable they can be shared.

Here we have three String objects, a, b and c, which happen to contain identical Strings. Because they are identical, the Java compiler only stores one copy of the String "hello".

```
String a = "hello";  
String b = "hello";  
String c = "hello";  
a, b and c are merely pointers to the same space.
```

Once the compiler discovers that the String object is the same for all three, it uses the same String for all three. Remember, a String can't be changed. In the following example, we are reassigning a reference, or making it point to a different place. We're not changing the String:

```
String a = "hello";  
String b = "hi";  
String c;  
c = a;  
c = b;
```

This helps explain what is happening when we compare two Strings in this manner: `if(a == b)`; We are comparing the references, not the objects.